Accelerating Just Image Transformers Through Efficient Numerical Sampling

Training-Free Inference Optimization with Existing Numerical Solvers

Iason-Daniil Zakynthinos

School of ECE, NTUA · Advanced Topics in Artificial Intelligence · 2026–2027

***Abstract*—Just image Transformers (JiT) generate images directly in pixel space, but sampling requires repeated evaluations of a large neural network. This project studies whether existing numerical solvers and fixed timestep schedules can reduce generation latency without retraining. Using one pretrained JiT-B/16 checkpoint at 256 × 256 resolution, I compare Euler, Heun, and second-order Adams-Bashforth sampling under controlled network-evaluation budgets. Uniform and nonuniform schedules are evaluated separately to isolate the effect of timestep placement. The primary outcome is the image-quality versus latency frontier. An initial target is a 2× reduction in latency while keeping FID within 10% of the reference sampler; this is an experimental objective, not an assumed result. The study will deliver reproducible sampler implementations, measurements, and visual comparisons.**

I. RESEARCH QUESTION AND HYPOTHESES

How much can pretrained JiT inference be accelerated through numerical solvers and timestep scheduling without substantially degrading generated image quality?

H1 (solver efficiency). A second-order multistep method can provide a better quality-latency trade-off than the reference Heun sampler by reusing previous velocity evaluations instead of computing a new correction at every step. The comparison is made at matched realized network-evaluation budgets and measured latency, rather than nominal solver steps.

H2 (timestep placement). A fixed nonuniform schedule can improve image quality relative to uniform spacing under the same solver and computational budget. Schedule parameters are tuned on separate validation seeds and frozen before final evaluation.

Success criterion. Seek a configuration with latency at most half that of the reproduced reference and FID no greater than 1.10 times its FID, using an identical evaluation protocol. Failure to meet this target is still informative: the final result will quantify the quality cost of faster sampling and identify the best measured operating points.

II. MOTIVATION AND RELATED WORK

JiT predicts clean images directly from noisy pixel-space inputs, avoiding a separately trained latent autoencoder [1]. Its simple architecture still requires many transformer evaluations to turn noise into an image. Numerical sampling therefore offers an inference-only optimization opportunity without changing the checkpoint or architecture.

Euler uses one velocity evaluation per update. Heun usually uses a second evaluation to correct an Euler prediction, improving numerical accuracy at additional cost. A multistep method instead reuses the preceding velocity to obtain a higher-order update with only one new evaluation after startup. Fewer evaluations can reduce latency, although overhead and guidance mean the relationship must be measured.

DPM-Solver++ demonstrates the value of efficient high-order integration for guided diffusion sampling [2]. Align Your Steps shows that timestep placement also affects sampling efficiency [3]. These works motivate the study; the project does not claim a new numerical solver or direct transfer of their reported gains to JiT. Its contribution is a controlled empirical comparison on a pretrained pixel-space transformer.

III. METHODOLOGY

Model and implementation. Use the public JiT-B/16 checkpoint and PyTorch implementation [4] for class-conditional ImageNet generation at 256 × 256 pixels. Experiments run on an NVIDIA RTX 5080, subject to initial compatibility, throughput, and memory checks. Model weights, architecture, and training remain unchanged. Record the checkpoint, code revision, software versions, precision, batch size, guidance settings, labels, and seeds.

Velocity construction. For noisy state z at time t and predicted clean image x_{pred}, follow the repository velocity parameterization:

$$v(z, t) = (x_{pred}(z, t) - z) / max(1 - t, \varepsilon)$$

The endpoint convention and denominator safeguard are fixed across methods. The original repository sampler is retained as a separate reference with its exact settings and measured evaluation count.

Euler. With $h_n = t_{n+1} - t_n$, the first-order baseline is

$$z_{n+1} = z_n + h_n v_n$$

Heun. Form $z^* = z_n + h_n v_n$, evaluate v^* at (z^*, t_{n+1}), and correct:

$$z_{n+1} = z_n + (h_n/2)(v_n + v^*)$$

Preserve and document the reference implementation's final Euler step so that endpoint handling is not a hidden source of differences.

Adams-Bashforth 2. Start with an Euler update, then reuse the previous velocity. For $r_n = h_n / h_{n-1}$, use the variable-step update

$$z_{n+1} = z_n + h_n[(1 + r_n/2)v_n - (r_n/2)v_{n-1}]$$

For constant steps this reduces to the standard second-order Adams-Bashforth method. Validate the implementation on simple test ODEs, check startup and variable-step coefficients,

and inspect stability under coarse schedules. Higher numerical order does not guarantee better generated images.

Schedules and comparisons. Compare uniform spacing with a small family of parameterized, monotone nonuniform grids. Tune on validation seeds and keep the selected grid fixed across test images. First compare solvers on uniform grids; then compare schedules with the solver fixed; finally compare the best combination with the unmodified JiT reference. Image-specific adaptive stepping is outside scope.

IV. EVALUATION, FEASIBILITY, AND EXPECTED CONTRIBUTION

Fair budgets. Evaluate nominal budgets of 10, 20, 30, 50, and 100 network passes. Count actual transformer evaluations, including both classifier-free guidance branches, solver substages, and startup. Log guided velocity evaluations separately. If an exact budget is infeasible for a method, use the nearest feasible configuration at or below it and report its realized count. Hold labels, initial noise, guidance, and evaluation settings constant across methods.

Image quality. Measure Fréchet Inception Distance (FID) against fixed ImageNet reference statistics. Use small generated sets only to screen configurations, then evaluate finalists with the same larger sample count and preprocessing. Set that count after measuring pilot throughput and report it explicitly. Repeat finalist evaluations where feasible and report variability; small differences in noisy FID estimates are not evidence of superiority. Schedules are never tuned on final evaluation seeds.

Runtime and memory. Measure synchronized wall-clock generation time after warm-up, at fixed precision and batch size, with repeated runs. Report median latency, its variability, actual network passes, and peak GPU memory. Exclude model loading and disk I/O from generation timing and state this boundary. Network-pass reductions are not treated as measured speedups.

Analysis. Plot FID against measured latency and report Pareto-efficient configurations: those for which no alternative is both faster and at least as good in FID. Compare paired images generated from identical noise and labels, and selected intermediate clean-image predictions. These visualizations explain differences but do not replace distribution-level evaluation.

Feasibility. Limit the study to one pretrained model and inference-only changes. Pilot runs establish throughput and the final compute budget before large-scale generation. If resources are insufficient, prioritize solver comparisons and reduce schedule variants and budget levels. Preserve the reference and equal-sample final comparisons; do not assume an unmeasured GPU-hour estimate.

Expected contribution. A reproducible evaluation of existing numerical sampling methods for JiT, a recommended configuration for different computational budgets, and a measured account of the quality-latency trade-off. Deliverables include sampler code, experiment settings, quantitative benchmarks, paired image grids, and a final technical report. Negative results and instability at very low budgets will be reported alongside successful configurations.

V. PROPOSED SCHEDULE

Weeks 1–2: set up JiT, check hardware, reproduce reference samples, and benchmark throughput. *3–4:* implement and validate Euler, Heun, and multistep sampling. *5–6:* compare solvers at fixed network-pass budgets. *7–8:* implement and tune nonuniform schedules on validation seeds. *9–10:* evaluate selected configurations using larger equal-sized image sets. *11:* analyze quality-latency frontiers and visual comparisons. *12–13:* prepare reproducible code, final report, and presentation.

REFERENCES

[1] T. Li and K. He, “Back to Basics: Let Denoising Generative Models Denoise,” arXiv:2511.13720, 2025.

[2] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, “DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models,” arXiv:2211.01095, 2022.

[3] A. Sabour, S. Fidler, and K. Kreis, “Align Your Steps: Optimizing Sampling Schedules in Diffusion Models,” in Proc. ICML, 2024.

[4] T. Li, “JiT: PyTorch implementation,” github.com/LTH14/JiT. Accessed Nov. 5, 2026.